PRANAV DATTATRAY BHONGALE

+91 8459231239 | pranavbhongale18@gmail.com | LinkedIn | GitHub | Pune, Maharashtra, India

SUMMARY

Electronics Engineering undergraduate (CGPA: **9.26/10**) specializing in VLSI front-end design and computer architecture. Proven capability in microarchitectural development, featuring an RV64 5-stage pipelined processor with dynamic branch prediction (2-bit BPU, 16-entry BTB), a **512 KB non-blocking cache**, and a product-grade NPU for ADAS systems. Proficient in **SystemVerilog RTL design** and data-driven performance optimization, complemented by advanced case studies on complex processors and accelerators (Snapdragon 730G, AMD MI300, Tomahawk 6).

EDUCATION

Walchand College of Engineering, Sangli *2024 – 2028*

B.Tech in Electronics Engineering | 3rd Year | CGPA: **9.26 / 10.0**

Relevant Coursework: Digital Design (Prof. Frank Gürkaynak, ETH Zürich); Computer Architecture (Prof. Onur Mutlu, ETH Zürich, SAFARI Research Group); VLSI Flow – RTL to GDSII (Prof. Adam Teman, Bar-Ilan University)

PROJECT EXPERIENCE

NPU for ADAS – Product-Based Neural Processing Unit **GitHub** *2024 – Present [In Progress]*

SystemVerilog | *RTL Design* | *Domain-Specific Accelerator*

- Architecting a systolic-array NPU for ADAS workloads with on-chip SRAM buffering, output-stationary dataflow, and INT8 and FP8 quantization, targeting minimum data movement under timing, area, and bandwidth constraints
- Analysing activation-function hardware (special-function unit) overhead vs. benefit using Amdahl's Law and FP8 systolic-array sizing from ADAS workload GEMM shapes

RISC-V RV64 5-Stage Pipelined Processor GitHub

SystemVerilog | *C++* | *Computer Architecture*

- Designed a 5-stage RV64I+M pipeline with a 2-bit branch predictor and 16-entry Branch Target Buffer (BTB), including misprediction flush and stall logic in the PC-generation unit
- load-use hazard detection with stalling; built a C++ pipeline model with valid/ready handshakes
- Verified by hierarchical testbench flow in with reference to the golden model (Verilator, GTKWave)

512 KB Non-Blocking Cache Memory GitHub

SystemVerilog | *RTL Design* | *Memory Hierarchy*

- Designed a 4-way, write-back, write-allocate, 64 B-line non-blocking cache using MSHRs to support concurrent misses and improve memory-level parallelism
- Built tag/data arrays, valid & dirty bits, and SRRIP replacement; evaluated hit/miss latency and CPI impact against a blocking design (extended from an 8 KB blocking cache)

CASE STUDIES & TECHNICAL STUDY

- **Qualcomm Snapdragon 730G:** SoC subsystem study covering CPU, GPU, AI/DSP, ISP and interconnect
- **AMD Instinct MI300:** chiplet architecture, CDNA3 compute dies, HBM3 unified memory, Infinity Fabric/Cache and coherence
- **Network-on-Chip:** routing (XY, west-first, odd-even), arbitration (WRR, DRR, iSLIP) and virtual-channel deadlock Avoidance , communication protocall like PCIe , UCIe , UALink , AXI series
- **RTL-to-GDSII Flow:** complete ASIC implementation flow, following Prof. Adam Teman's course (Bar-Ilan University)

TECHNICAL SKILLS

HDL / RTL: SystemVerilog, SystemC (basic)

Digital Design: FSM Design, Pipelining, Clock Domain Crossing, Timing Analysis, Clock gating

Computer Architecture: RISC-V ISA, Cache Hierarchy (Blocking/Non-Blocking), Branch Prediction, CPI Analysis, NoC, Out-of-Order & Superscalar, SoC, NPU/Systolic Arrays

Verification: Testbench architecture, UVM (basic)

Tools & Languages: C++, Python, Verilator, GTKWave, Git/GitHub